Toward a Clean MeTTa-Pure: Kernel, Profile, Bridge, and Runtime Architecture

Conceptual Primer and Design Record

Prepared as a technical primer for future MeTTa work

2026-03-04

Abstract

This note consolidates a design program for a clean dependent-type-theoretic core for MeTTa together with its relation to the broader MeTTa family of runtimes, profiles, and modal semantics. The central claim is that a satisfactory architecture requires a small trusted kernel, a MeTTa-IL presentation of that kernel, and an explicit bridge between the two. The note explains why a direct attempt to treat the ambient MeTTa-IL host syntax as the kernel theorem domain creates avoidable proof pathologies, why a separate scoped kernel is the right long-term foundation, why one should keep both executable and proof-theoretic semantics, and how all of this can still define a bona fide form of MeTTa rather than an unrelated theorem prover. The paper also records the recommended package structure for Lean and Lean-backed runtimes, the semantics of profiles such as HE and PeTTa, and a least-regret roadmap for the remaining bridge theorems.

Contents

1 Purpose and scope **1**

2 Diagnosis: what is wrong with the naive picture? **1**

3 Three distinct objects: kernel, presentation, and runtime family **2**
- 3.1 The trusted kernel . . . 2
- 3.2 The MeTTa-IL presentation . . . 2
- 3.3 The runtime family . . . 3

4 The central thesis: MeTTa-Pure is not enough by itself **3**

5 The A/B/C semantic stratification **3**
- 5.1 A: operational/profile-facing reduction . . . 4
- 5.2 B: full kernel theory reduction . . . 4
- 5.3 C: profile-side theory closure . . . 4

6 Why the old bridge failed **4**

7 The correct bridge: contextual quotation into locally nameless syntax **5**
- 7.1 Why this matters . . . 5

8 The bridge theorem should be about substitution, not about `inst0` **5**

9 Why a separate kernel syntax is the right long-term route **6**

10 MeTTa-IL as operational modal base **6**

11 Initiality and freeness: the right conjectures **6**
11.1 For the kernel . . . 6
11.2 For MeTTa-IL . . . 7

12 Profiles, not implementations, are the right semantic comparison unit **7**

13 Pure as a bona fide MeTTa profile **7**

14 The right package structure **8**

15 Simple runtime, staged runtime, and external kernels **8**
15.1 Simple runtime . . . 8
15.2 Staged runtime . . . 8
15.3 Optional low-level kernels . . . 8

16 Optional DSL profiles and theorem-backed lowering **8**

17 What should be considered locked in? **9**

18 Open problems worth keeping in scope **9**

19 A concise roadmap for future implementation work **10**

20 Conclusion **10**

1 Purpose and scope

The goal is to articulate a clean conceptual design for *MeTTa-Pure*: a mathematically trustworthy dependent type theory that remains genuinely connected to the larger MeTTa ecosystem rather than drifting into an isolated proof assistant core. The emphasis here is architectural and semantic rather than historical. The note is intended to be useful in three ways:

1. as a primer for future discussions about MeTTa-Pure, MeTTa-Core, and profile design,
2. as a conceptual reference for the ongoing Lean formalization, and
3. as a stable record of the design choices that should not be re-litigated without a strong mathematical reason.

The text is intentionally didactic. It does not attempt to present polished final metatheory; instead, it organizes what has already been learned and identifies the correct abstractions for future work.

2 Diagnosis: what is wrong with the naive picture?

A first temptation is to say: current MeTTa already has a type judgment, already has rewriting, and already has a generic host syntax, so perhaps a dependent type theory can simply be layered

onto the current structure. This is the wrong framing.

The core diagnosis is:

- The existing MeTTa runtime typing discipline is not a proof-assistant kernel. In the formalized development, the existing `HasType` behaves essentially as annotation lookup. That is useful operationally, but it is not enough for dependent type theory because it is not closed under the conversion behavior needed for kernel reasoning.
- MeTTa-IL itself is not the problem. It is a strong operational and categorical substrate: it gives a shared `Pattern` syntax, `LanguageDef`-driven rewriting, executable and declarative engine relations, and the derived modal/behavioral layer used by OSLF.
- The real mistake is to identify the *ambient host representation* with the *kernel theorem domain*. A host datatype may be excellent for interoperation, runtime lowering, and modal analysis while still being the wrong domain on which to prove confluence, injectivity, or subject reduction for a small DTT kernel.

This diagnosis yields the central design principle of the project:

> **Kernel metatheory should be proved on the least syntax closed under kernel operations, not on the ambient host representation.**

That principle is the hinge on which the rest of the design turns.

3 Three distinct objects: kernel, presentation, and runtime family

The clean design separates three objects that were initially blurred together.

3.1 The trusted kernel

The first object is a scoped dependent-type-theoretic kernel, here called *PureKernel*. It should carry:

- its own syntax (for example, `PureTm` indexed by scope depth),
- renaming and substitution operations,
- one-step reduction and its reflexive-transitive closure,
- confluence or Church–Rosser style theorems,
- conversion-sensitive typing, and
- subject reduction.

This kernel is the trusted mathematical core. It is where one proves the theorems that matter for a DTT.

3.2 The MeTTa-IL presentation

The second object is a *Pattern-side presentation* of the same pure fragment as a `LanguageDef`, here called `mettaPure`. This presentation is not the theorem source of truth; instead, it is the way the pure kernel appears *inside the broader MeTTa world*. It has:

- the same intended constructors (universes, Π, Σ, identity, lambda, application, pair, projections, refl),

- a small set of rewrite rules corresponding to the computational core (currently the three beta rules), and
- automatic access to the generic MeTTa-IL machinery such as constructor categories and OSLF synthesis.

The presentation is therefore not an arbitrary encoding. It is the *MeTTa profile* of the kernel.

3.3 The runtime family

The third object is the family of runtime profiles over the shared substrate. The key abstraction here is not "one master semantics for everything" but a *profile interface*.

A good conceptual shape is:

```
MeTTaCoreProfile = LanguageDef + premises + runtime policy + optional state
constructor + profile metadata.
```

Under this view:

- the pure profile is one profile,
- HE is another,
- PeTTa is another,
- fuller stateful MeTTa profiles are yet others.

This profile notion is the right place to say what makes something a *bona fide* form of MeTTa.

4 The central thesis: MeTTa-Pure is not enough by itself

A bare DTT kernel is not yet "a form of MeTTa." It becomes one only if all three of the following are present:

1. a trusted kernel with its own theorems,
2. a MeTTa-IL presentation/profile of that kernel, and
3. a conservative bridge theorem connecting the kernel to the profile.

This may be summarized as:

> **MeTTa-Pure is bona fide MeTTa if and only if it is a conservative core profile with a proved embedding from the trusted kernel into the shared MeTTa runtime substrate.**

This is a stronger and more useful statement than saying "the syntax looks similar" or "it can be encoded somehow."

5 The A/B/C semantic stratification

One of the most important insights is that there are not two but *three* semantic layers that should be kept separate.

5.1 A: operational/profile-facing reduction

Layer A is the executable or profile-facing reduction relation. It is the fragment that can be soundly interpreted directly by the current profile engine. In the present development this is a deliberately small, closed fragment: a few basic beta steps for closed pure terms.

A is useful because it gives immediate execution and concrete runtime tests.

5.2 B: full kernel theory reduction

Layer B is the actual DTT kernel reduction. It includes the congruence closure needed for confluence, injectivity, and subject reduction. This is the true proof-theoretic relation of MeTTa-Pure.

B is not reducible to A. If one weakens B to match A too early, the kernel loses the very structure that makes a DTT mathematically useful.

5.3 C: profile-side theory closure

Layer C is the profile-side analogue of B. It lives not on kernel terms but on quoted Pattern terms inside the pure MeTTa profile. It is needed because B does not map directly to raw top-level profile one-step reduction.

The correct form of C is an *intrinsic contextual closure* on the profile side, not merely a transported existence statement.

In short:

- A is for execution,
- B is for kernel proof theory,
- C is for relating B to the profile world without pretending the profile's raw one-step semantics already performs all kernel congruence steps.

The key architectural lesson is:

> **Do not choose between A and B. Keep A for execution, keep B for theory, and define C so that B has a mathematically honest home on the profile side.**

6 Why the old bridge failed

The earlier bridge attempts ran into a recurring obstacle around beta reduction under binders. The root cause was not merely a difficult proof state. It was a bad bridge design.

A naive quotation from scoped kernel terms into `Pattern` sent kernel variables directly to `bvar` indices in the host representation. That makes the bridge too close to the host internals and breaks exactly where beta and substitution matter most.

The correct diagnosis is:

> **The failure came from using the wrong quotation, not from any impossibility of a faithful bridge.**

The right quotation must be a *contextual locally nameless quotation*.

7 The correct bridge: contextual quotation into locally nameless syntax

The bridge should not be a raw map

$$\texttt{PureTm n} \to \texttt{Pattern}$$

with no environment. It should instead be parameterized by a naming environment:

$$\texttt{quoteTmWith} : (\texttt{Fin n} \to \texttt{String}) \to \texttt{PureTm n} \to \texttt{Pattern}.$$

The intended behavior is:

- kernel variables become `fvar`s carrying names from the environment,
- when descending under a binder, one chooses a fresh binder name,
- the body is quoted with the extended environment,
- and then the chosen free variable is closed using `closeFVar` to form the locally nameless host representation.

This is the standard way to bridge scoped syntax and locally nameless syntax.

7.1 Why this matters

With this quotation, the central beta bridge is no longer a raw claim about host opening. Instead, one can prove a semantic theorem in the style:

> quotation commutes with kernel substitution after translating the substitution into a host-side substitution environment.

That is the right theorem. The earlier approach tried to prove a special-case `inst0` statement too early and therefore kept running into binder-level weakening and lifting mismatches.

8 The bridge theorem should be about substitution, not about `inst0`

A major source of wasted effort was attempting to prove a specialized bridge theorem for the single-substitution operator `inst0` before proving the general substitution theorem.

The correct order is:

1. prove a generic renaming bridge,
2. prove a generic substitution bridge,
3. derive the `inst0` case as a corollary,
4. and only then refactor the embedding theorems so the assumption wrappers disappear.

Conceptually, one wants a theorem of the following form:

> `quoteTmWith` of a kernel substitution equals host-side application of a quoted substitution environment to the quoted term.

Once that is proved, all the special cases become easy. Without it, every binder case must be re-solved by hand.

9 Why a separate kernel syntax is the right long-term route

There was a fork between:

- continuing to prove confluence and substitution transport directly on ambient `Pattern`, while restricting theorem domains via predicates such as `PureTmPattern`, or
- introducing a dedicated scoped kernel syntax.

The second route is the cleaner long-term choice.

The reason is simple: if the kernel is meant to be trusted, then its theorems should not depend on ambient constructors that belong to the host runtime substrate but not to the kernel itself. The correct abstraction boundary is:

> **kernel syntax for kernel theorems; host syntax for quotation, runtime interoperation, and profile presentation.**

The earlier `PureTmPattern` restriction was an important diagnostic step, but the dedicated scoped syntax is the final answer.

10 MeTTa-IL as operational modal base

Another central insight is that the runtime/modal side and the kernel/type-theoretic side are complementary rather than competing.

MeTTa-IL already behaves like an operational indexed-modal doctrine:

- `LanguageDef` gives a generic operational presentation,
- the MeTTa-IL engine gives declarative and executable rewriting,
- OSLF synthesizes modal/behavioral types from the reduction structure,
- and the resulting setup supports adjoint modal reasoning such as a derived $\Diamond \dashv \Box$ pattern.

This does *not* by itself determine the internal DTT of the pure mode. The pure mode must still be given suitable comprehension or CwF-like structure. But it does provide the right operational base on which the pure profile can live.

Thus:

> **MeTTa-IL supplies the operational and modal base; MeTTa-Pure supplies the trusted DTT kernel; the bridge connects them.**

11 Initiality and freeness: the right conjectures

Several universal-property conjectures naturally arise. The ones worth keeping are the restrained ones.

11.1 For the kernel

A good conjecture is:

> **MeTTa-Pure is initial among admissible pure kernels with the chosen MeTTa-like signature and a faithful interpretation into the MeTTa-IL profile world.**

This captures the idea that the pure kernel is not arbitrary; it is the minimal honest DTT core compatible with the intended MeTTa presentation discipline.

11.2 For MeTTa-IL

A better conjecture than "MeTTa-IL is initial among all GSLTs" is something like:

> **MeTTa-IL is free or initial among operational GSLT presentations generated from the intended MeTTa signature, binder forms, collections, premises, and congruence policy.**

The point is not to force an over-large universality claim but to find the smallest category in which the natural universal property is true.

12 Profiles, not implementations, are the right semantic comparison unit

A recurring source of confusion is the urge to define the theory in terms of today's implementation snapshots. That is the wrong way around.

HE, PeTTa, and future MeTTa variants should be treated as *profiles* over a shared substrate rather than as sacred operational facts.

This yields several benefits:

- one can freeze a profile and compare implementations against it,
- one can let the pure profile remain mathematically ideal while runtime approximations evolve,
- and one can compare profiles without forcing them into one monolithic semantics.

This is especially important because current HE, PeTTa, and broader MeTTa-Core profiles need not be identical and need not evolve in lockstep.

13 Pure as a bona fide MeTTa profile

The core worry is legitimate: how can one know that the new DTT kernel is really part of MeTTa rather than just some imported theorem prover?

The answer is not rhetorical but formal.

A pure kernel becomes a bona fide MeTTa form when:

1. it is packaged as a `MeTTaCoreProfile`,
2. its quoted closed terms are legal Pattern terms for that profile,
3. kernel steps soundly embed into the profile's theory-side semantics,
4. and the embedding is conservative on the quoted image.

That is the right criterion. In particular, a pure kernel is not required to look operationally like HE or PeTTa in every internal detail. It is required to sit inside the same family architecture with the right bridges.

14 The right package structure

Another important insight concerns the Lean codebase itself.

A single monolithic package is the wrong shape if one wants:

- a lightweight runtime core with Batteries-only dependencies,
- a heavier Mathlib-based proof layer,
- and shared executable types and engines between the two.

The recommended layout is a monorepo with multiple Lean packages:

- `mettail-core`: shared executable substrate (Pattern, LanguageDef, matching, core engine, profile data),
- `algorithms`: lightweight runtime implementations and sessions,
- `mettapedia`: Mathlib-heavy proof, specification, and conformance layer.

This avoids both type duplication and a transitive dependency from runtimes into Mathlib.

15 Simple runtime, staged runtime, and external kernels

The runtime story also has a clean three-stage shape.

15.1 Simple runtime

The simple runtime should be a direct interpreter over a frozen profile bundle. It should be easy to test against corpora and easy to relate to declarative semantics.

15.2 Staged runtime

A staged Lean runtime should be a refinement of the same profile bundle, not a separate semantics. The idea is to specialize matching and dispatch while preserving the reference meaning.

15.3 Optional low-level kernels

Only after profiling should one consider lower-level verified kernels (for example via C/LLVM pathways or verified external code). These should refine the same reference semantics rather than replacing it.

The same principle applies throughout:

> **one semantic source of truth, many execution modes.**

16 Optional DSL profiles and theorem-backed lowering

The TensorDSL work provided an important architectural template.

A domain-specific notation should generally not mutate the core parser or runtime. Instead it should be handled as:

1. a typed profile-specific AST,
2. profile-specific invariants and proofs,

3. a verified lowering into the shared Pattern substrate,
4. and a cross-runtime corpus validating the lowering against multiple backends.

This template should be reused. It is a clean way to build rich domain-specific layers on top of a stable MeTTa core.

17 What should be considered locked in?

The following decisions are strong candidates for being treated as stable architectural commitments.

(i) Separate kernel syntax

Kernel metatheory should remain on a scoped kernel syntax rather than on raw ambient Pattern terms.

(ii) Profile architecture

Pure, HE, full-core, and similar variants should be treated as profiles over a common substrate.

(iii) A/B/C semantics

Execution, kernel theory, and profile-side theory closure should remain distinct layers.

(iv) Contextual quotation

The semantic bridge should use contextual locally nameless quotation rather than raw de Bruijn-like embedding into the host.

(v) Substitution before `inst0`

The fundamental bridge theorem should be a general quotation/substitution theorem. Any `inst0` theorem should be derived from it.

(vi) Direct lowering for kernel execution

Runtime execution of the kernel should go through direct lowering from the kernel AST to the pure profile, not through a generic text parser as the correctness anchor.

(vii) `quoteRaw` is debug-only

A naive raw quotation is useful as a negative regression witness, not as a semantic bridge.

18 Open problems worth keeping in scope

Several major tasks remain, but they should be pursued in the order implied by the architecture above.

1. Finish the generic quotation/substitution theorem.
2. Remove bridge assumptions and make the B→C theorems unconditional.

3. Finalize the C1 profile-theory layer and relate it cleanly to profile reduction closures.
4. Decide the scope of algorithmic type checking for the kernel and whether to provide a complete checker or leave this explicitly as future work.
5. Broaden the pure profile's operational fragment A only after the semantic stratification is complete.
6. Continue unifying the Lean runtime and Rust runtime around frozen profile artifacts.

19 A concise roadmap for future implementation work

A good near-term sequence is:

1. complete the generic renaming bridge,
2. complete the generic substitution bridge,
3. derive unconditional `inst0` corollaries,
4. remove "assuming_inst0" wrappers,
5. prove unconditional B→C soundness,
6. then, and only then, resume broader runtime/profile refinements.

For the runtime side, the recommended order remains:

1. simple reference runtime over a frozen profile bundle,
2. staged/specialized runtime as a refinement,
3. optional lower-level verified kernels only after profiling justifies them.

20 Conclusion

The main conceptual achievement is not merely a collection of proofs. It is a clarified architecture.

The mature picture is this:

- **PureKernel** is the trusted DTT core.
- **`mettaPure`** is its MeTTa-IL presentation as a profile.
- **PatternBridge** is the semantic embedding from kernel to profile.
- **A/B/C** separate execution, proof theory, and profile-side theory semantics.
- **MeTTa-IL** remains the shared operational and modal base.
- **HE, PeTTa, and fuller MeTTa variants** remain comparable profiles rather than competing foundations.

This resolves the central tension. One does not need to choose between a mathematically clean kernel and a genuine MeTTa-family language. The correct design is to have both, connected by explicit profile and bridge structure.

That is the scaffolding worth preserving.

Appendix: minimal glossary

PureKernel The trusted scoped DTT kernel with its own syntax, substitution, reduction, typing, and subject reduction.

`mettaPure` The Pattern-side `LanguageDef` presentation of the pure kernel inside MeTTa-IL.

MeTTa-Core The shared runtime/profile family architecture over `LanguageDef` and premise bundles.

Profile A frozen semantic/runtime bundle describing one MeTTa-family variant (for example Pure, HE, or full-core).

A/B/C The three semantic layers: executable fragment, kernel theory, and profile-side theory closure.

C1 The intrinsic contextual closure on the profile side corresponding to the kernel theory relation.

Contextual quotation The environment-aware locally nameless map from kernel terms to Pattern terms.

OSLF The modal/behavioral layer synthesized from the operational structure of a language/profile.